

Funkce a procedury

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická v Jihlavě

© Karel Richta, 2020

Základy strukturovaného programování, ZSP
11/2020, Lekce 7

<https://moodle.vspj.cz/course/view.php?id=200871>



Funkce a procedury

- Tvorba programů se podobá jiným tvůrčím aktivitám při řešení problémů. Od jisté úrovně složitosti problému je řešení netriviální a mentálně jako celek nejprve neuchopitelné.
- Platí známé pravidlo 7 ± 2 (tzv. Millerovo číslo odkazující na velmi známou a citovanou práci z oboru kognitivní psychologie, kterou napsal v roce 1956 George Armitage Miller z Princetonské university, a která odhaduje rozsah lidské krátkodobé paměti a popisuje, že počet prvků, které člověk udrží v krátkodobé paměti, je 7 ± 2 prvků).
- Složitější artefakty je tedy třeba řešit tak, že je rozdělíme na jednodušší tak, aby z nich šlo složit celkové řešení. V podstatě jde o řešení problémů technikou „rozděl a panuj“.

Základní programátorské pravidlo

- Pro řešení složitějších problémů proto používáme základní programátorské pravidlo:

Nevíš-li co dělat, zaveď proceduru (funkci, metodu)!

- Řešený problém rozdělíme na jednodušší podproblémy, které řeší takto zavedená procedura (funkce, metoda).
- Předpokládáme, že tyto podproblémy umíme vyřešit a z jejich řešení složit i řešení celého problému.
- Posléze dořešíme také podproblémy (procedury, funkce, metody).
- Často se při řešení podproblému odkazujeme rekurzivně na podobný problém, ale menšího rozsahu.

Příklad: Telefonní seznam

- Problém: Chceme vypsát telefonní seznam jako posloupnost záznamů o osobách, které jsou identifikovány pomocí čísel.
- Otázka: Umíme vypsát záznam o osobě? Neumíme → zavedeme si proceduru **vypis**, které předáme číslo osoby jako parametr a ona vypíše požadované údaje:

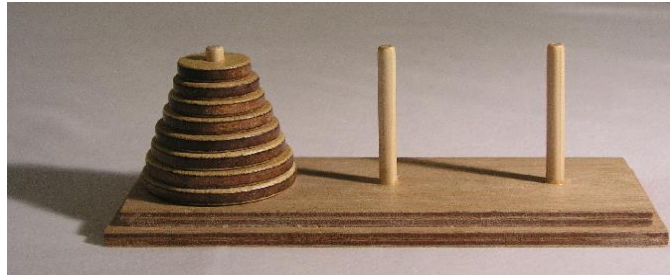
```
void vypis(int cislo);
```

- Hlavní program pak použije toto řešení (proceduru) a vypíše obsah telefonního seznamu např. cyklem:

```
for (auto i=0; i<MAX; i++) vypis(i);
```

Příklad: Hanojské věže

- Problém Hanojských věží - přesun N disků z tyče A na tyč B pomocí tyče C.



- Jednoduché řešení se zdá být prohlížení stavového prostoru úlohy (zkoušení různých tahů) a nakonec výběr správného řešení.
- Není to ale tak prosté – stavový prostor rychle nabývá na rozsahu. V prvním tahu máme 2 možnosti, ve druhém 4 atd.

Vypráví se legenda, že někde ve Vietnamu nebo Indii stojí klášter nebo chrám, v němž jsou hanojské věže se 64 zlatými kotouči.

Mniši každý den v poledne za zvuku zvonů slavnostně přemístí jeden kotouč.

V okamžiku, kdy bude přemístěn poslední kotouč, nastane konec světa.

Vyřešení tohoto hlavolamu pro 64 kotoučů však vyžaduje $2^{64}-1 = 18\,446\,744\,073\,709\,551\,615$ tahů,

takže i kdyby mniši stihli provést jeden tah každou sekundu

(a postupovali nejkratším možným způsobem), trvalo by jim vyřešení celého hlavolamu přibližně 600 miliard let.

Příklad: Hanojské věže (pokr.)

- Nevíme, jak přesunout disky – zavedeme si proceduru „*Přesuň N disků z tyče A na tyč B pomocí tyče C* “.
- Jednoduché řešení pomocí techniky rozděl a panuj – pro přesun N disků z tyče A na tyč B pomocí tyče C postupuj následovně:
 - *Přesuň $N-1$ disků z tyče A na tyč C pomocí tyče B* (rekurzivní volání procedury pro řešení stejného problému, ale pro $N-1$ disků).
 - *Přesuň jeden disk z tyče A na tyč B* (triviální podproblém – přesun největšího disku).
 - *Přesuň $N-1$ disků z tyče C na tyč B pomocí tyče A* (rekurzivní volání procedury pro řešení stejného problému, ale pro $N-1$ disků).
- Jednoduché rekurzivní řešení založené na vlastním principu problému. Jiná sofistikovanější řešení nejsou tak úspěšná.
- http://www.mathsisfun.com/games/hanoi_solver.html
- [Řešení na YouTube](#)

```

/* procedura presun */
void presun(int n, int x, int y, int z) {
    if (n < 1) return;
    else {
        presun(n - 1, x, z, y);
        printf("Move disc from %d to %d\n", x, y);
        presun(n - 1, z, y, x);
    }
};

int main(void) {
    int N;
    enum { A = 1, B, C };
    printf("Hanoi Towers\n");
    printf("Type N: "); scanf("%d",&N);
    presun(N, A, B, C);
    printf("Finish\n");
    system("PAUSE");
    return 0;
}

```



Deklarace a definice funkce

- Funkce zakrývá kus opakovaně využívaného kódu, který je znovupoužit při volání funkce.
- **Deklarací funkce** se deklaruje identifikátor funkce, typ výsledku a případné typy parametrů.
- Tělo funkce je dáno **definicí funkce**.
- Příklad **deklarace**:

```
long Power(long x, int y);
```

- Příklad **definice**:

```
long Power(long x, int y)
{
    long result = 1;
    while (y-- >= 0) result *= x;
    return result;
}
```


Parametry funkce

- Parametry funkcí jsou nahrazovány **hodnotou**, případně **referencí** (jen C++).
- **Pořadí** vyhodnocení parametrů není normou jazyka definováno, určuje si ho implementace C/C++.
- **Výsledek** funkce může být hodnota skalárního typu (aritmetický typ nebo ukazatel), struktura, třída nebo sjednocení.
- Deklarace funkcí často umísťujeme do hlavičkových souborů, abychom uživateli umožnili jejich použití. Definice funkcí pak ukládáme do zdrojových souborů, které můžeme přeložit a uživateli předat jen přeložený relativní kód.

Deklarace funkce (ANSI styl)

- **ANSI styl** - funkční prototyp (lépe funkční profil) - deklarace obsahuje jméno funkce, specifikaci typu výsledku, specifikace typů a případně i jmen parametrů (jména však nejsou významná):

```
int F(void); /* funkce bez parametrů */  
void G(long x, int y); /* procedura */  
char *H(char *, char *); /* bez jmen parametrů */
```

- Každý parametr je deklarován zvlášť.
- Typ návratové hodnoty **void** = funkce nevrací hodnotu (procedura).
- **void** místo seznamu parametrů = funkce bez parametrů.
- Volání funkcí definovaných ANSI stylem se kontroluje a provádějí se případné konverze skutečných parametrů.

Příklad – použití prototypů

```
// použití funkčních prototypů
#include <iostream>
using namespace std;

void odd(int x); // prototyp funkce odd
void even(int x); // prototyp funkce even

int main() {
    int i;
    do {        cout << "Please, enter number (0 to exit): "; cin >> i;
                odd(i);
    } while (i != 0);
    return 0;
}

void odd(int x) {
    if ((x % 2) != 0) cout << "It is odd.\n"; else even(x);
}

void even(int x) {
    if ((x % 2) == 0) cout << "It is even.\n"; else odd(x);
}
```

Volání funkce

- Syntaxe:

$F (X1, X2, \dots Xn)$

- kde F je ukazatel na funkci a $X1, \dots Xn$ jsou výrazy tvořící skutečné parametry volání.
- Identifikátor funkce je konstantním ukazatelem na funkci.
- Při volání funkce deklarované ANSI stylem se:
 - kontroluje počet parametrů a typy parametrů,
 - skutečné parametry se konvertují pomocí roztažení (promotion) a navíc typ **float** se konvertuje na **double**,
 - kontroluje přípustnost skutečných parametrů a provádějí případné konverze (podobně jako při přiřazení).
- Pozor: závorky jsou třeba, i když se jedná o funkci bez parametrů!
- V C může existovat v celém programu jen jedna funkce daného jména.
- V C++ může existovat více funkcí stejného jména, ale musí být rozlišitelné pomocí parametrů (tzv. přetížení identifikátorů funkce).

Příklady volání funkcí

```
long Power(long, int);  
long l; int a, b; float f;  
l = Power(l, 10);  
a = Power(a + 3, b);  
f = Power(f, 3);
```

```
void Swap(int *, int *);  
int x, y;  
Swap(&x, &y);
```

```
void CtiPole(int, int *);  
int pole[10];  
CtiPole(10, pole);
```

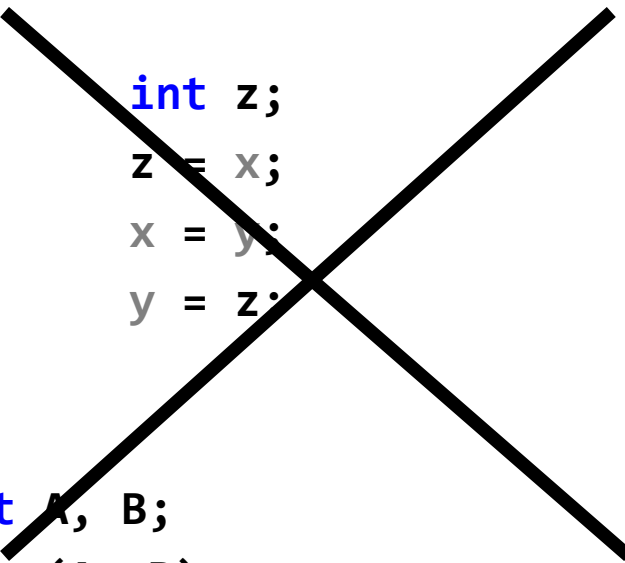
Parametry funkce volané hodnotou

- Volání hodnotou znamená, že jako skutečné parametry můžeme použít libovolný výraz převeditelný na daný typ. Výraz se vyhodnotí a jeho hodnota se uloží do formálního parametru, který funguje jako lokální proměnná.
- Funkce s parametry volanými hodnotou vrací nejvýše jeden výsledek – funkční hodnotu (zpravidla přes zásobník). Pokud vrací **void** – nevrací nic (je to procedura).
- Pokud chceme, aby funkce vracela více hodnot, musíme využít globální proměnné, nebo použít jako parametry ukazatele, nebo použít parametry volané referencí (viz dále).
- Přes parametry typu ukazatel můžeme manipulovat s objekty mimo lokální prostor těla funkce (podobně pomocí reference). To může vyžadovat určitou disciplinu.

Příklad: funkce, která prohodí 2 buňky

```
void swap(int x, int y)
{
    int z;
    z = x;
    x = y;
    y = z;
}

int A, B;
swap(A, B);
```



Chybně

```
void swap(int *x, int *y)
{
    int z;
    z = *x;
    *x = *y;
    *y = z;
}

int A, B;
swap(&A, &B);
```

Správně

Pořadí vyhodnocení parametrů

- Není normou jazyka definováno, implementace to může udělat tak, jak to vyjde efektivněji (pro procesory Intel zpravidla zprava doleva, pro procesory Sparc zleva doprava).
- Musíme programy psát tak, aby na tom nezáleželo – ve skutečných argumentech nepoužívat operace s vedlejšími efekty.

```
#include <stdio>

void foo(int x, int y) {}

int main() {
    foo(sprintf("foo"), sprintf("bar"));
    return 0;
}
```

Ve Windows vytiskne:
barfoo
Na Linuxu:
foobar

Cvičení: Pořadí vyhodnocení param.

```
#include <stdio.h>

void f(int x, int y, int z) {}

int p(int x)
{
    printf("Parametr: %d\n", x); return x;
}

int main()
{
    f(p(1), p(2), p(3));
    return 0;
}
```



Rekurzivní funkce

```
// factorial calculator
#include <iostream>

long factorial(long a) {
    if (a > 1)
        return (a * factorial(a - 1));
    else
        return 1;
}

int main() {
    long number = 9;
    std::cout << number << "! = " << factorial(number);
    return 0;
}
```

Parametry programu

- Funkce `main` může být bez parametrů:

```
int main(void) { ... }
```

- Funkce `main` může mít parametry:

```
int main(int počet, char *slova[]) { ... }
```

- 1.parametr udává počet slov v příkazovém řádku (slova jsou oddělena mezerou)
- 2.parametr je seznam slov

Příklad: při vyvolání programu `copy.exe`

```
>copy muj.txt tvuj.txt
```

dostane funkce `main` parametry:

```
počet == 3
```

```
slova[0] == "copy"
```

```
slova[2] == "tvuj.txt"
```

```
slova[1] == "muj.txt"
```

```
slova[3] == NULL (0)
```

Parametry programu (pokr.)

Příklad: správné řešení programu copy.exe s parametry:

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc != 3)
        fprintf(stderr,
            "Chybne volani copy, spravne: copy vstup vystup\n");
    else
        printf(
            "Spravne volani copy ve tvaru: copy %s %s\n", argv[1],
            argv[2]);

    return 0;
}
```

Návratová hodnota funkce `main`

- Pokud skončí hlavní program úspěšně, měla by být funkce `main` zakončena příkazem: `return 0;`
- Pokud funkce `main` skončí bez tohoto příkazu, předpokládá se úspěšný konec (tj. jakoby se provedl příkaz `return 0;`). To platí jen pro funkci `main` a případně funkci, která vrací hodnotu typu `void`. Někteří autoři nedoporučují implicitní `return 0;` jako špatnou praxi.
- Pokud vrátí funkce `main` hodnotu 0 (ať implicitně nebo explicitně), je to interpretováno jako úspěšný konec programu. Vracená hodnota může být zpravidla prostředím dále využita.
- Jiné hodnoty než 0 mohou být interpretovány jako příznak chyby. Zaručené hodnoty jsou definovány v knihovně `stdlib.h` (`cstdlib`):

Hodnota	Popis
0	úspěšný konec
EXIT_SUCCESS	úspěšný konec
EXIT_FAILURE	neúspěšný konec, chyba

Deklarace funkce s proměnným počtem parametrů

- Funkce s **proměnným počtem parametrů** (nutno použít standardní knihovnu `<stdarg.h>`)

```
#include <stdarg.h>
```

```
int K(int, ...); /* proměnný počet parametrů */
```

```
int printf(char *format, ...);
```

Definice funkce s proměnným počtem parametrů

- Hlavička: paměťová třída, typ návratové hodnoty, jméno funkce a seznam deklarací pevných parametrů zakončený ... (ellipsis):

```
#include <stdarg.h>
long Sum(int n, ... )
{
    long result = 0; va_list pt;
    va_start(pt, n);
    while (--n >= 0) result += va_arg(pt, int);
    va_end(pt);
    return result;
}
long l = Sum(4, 2, 45, 3, 17);
```

- Při volání funkcí s proměnným počtem parametrů se kontrolují pevné parametry, proměnné parametry se samozřejmě kontrolovat nedají.

Techniky návrhu algoritmů

- Při návrhu řešení nějakého problému se snažíme najít algoritmus, který daný problém řeší.
- Zpravidla lze rychle navrhnout naivní řešení, které přímo vychází z definice problému. Toto řešení ale zpravidla nebývá příliš efektivní – problém je zadán tak, aby jeho popis byl co nejjednodušší, naivní řešení toto zadání kopíruje.
- **Př.:** problém řazení posloupnosti čísel lze vyjádřit tak, že hledáme permutaci vstupní posloupnosti, která je seřazená (např. vzestupně). Naivní řešení tedy bude probírat všechny permutace vstupní posloupnosti a každou testovat, zda je seřazená. V nejhorším případě tedy najdeme správné řešení jako poslední a složitost řazení bude $O(n!)$.
- Pokud by byla vstupní data konečná a dostatečně malá, lze předpočítat všechny výstupy a odpovídající algoritmus by měl konstantní složitost $O(1)$.
- Ve skutečnosti jsou ale vstupní data zpravidla v principu nekonečná a pro návrh algoritmu musíme použít nějakou metodu založenou na rekurzi.

Rekurze

Rekurze je metoda zápisu algoritmu, kde se stejný postup opakuje na částech vstupních dat.

Výhody:

- úspora: zápis kódu je kratší a konečným zápisem lze definovat ∞ množinu dat,
- přirozené vyjádření: opakování a sebepodobnost jsou v přírodě běžné,
- intuitivní vyjádření: explicitně pojmenovává to, co se opakuje v menším,
- expresivní vyjádření: rekurzivní specifikace umožňuje poměrně snadnou analýzu složitosti a ověření správnosti.

Nevýhody:

- interpretace nebo provádění rekurzivního kódu používá systémový zásobník pro předání parametrů volání a proto vyžaduje systémovou paměť navíc,
- dynamická alokace systémového zásobníku a ukládání parametrů na něj navíc představuje také časovou režii.

Kvůli výpočetní úplnosti ale prakticky všechny dnešní programovací jazyky rekurzi povolují.

Technika rozděl a panuj

(divide-and-conquer, divide et impera)

- Algoritmy navrhované technikou rozděl a panuj mají tři fáze:
 - rozdělení problému na menší podproblémy (divide),
 - rekurzivní volání sama sebe na menší podproblémy (conquer) a
 - sestavení výsledného řešení z řešení podproblémů (combine).
- Jako každá rekurze, i algoritmy založené na technice **rozděl a panuj** musí někdy svůj rekurzivní sestup zastavit. Typicky se tak děje u triviálních velikostí problému.
- Dělení na podproblémy použijeme pro netriviální velikosti dat.
- Zpravidla lze rozlišit dělení na dělení **vyvážené**, kdy se vstupní data snažíme rozdělit na několik velikostí vyvážených podskupin, či **nevyvážené**, kdy se podskupiny liší svou velikostí.
- Další rozlišení technik rozděl a panuj lze charakterizovat podle důrazu kladeného buď na **analýzu** vstupních dat (divide), nebo na **syntézu** výstupu (combine).

Př.: Řešení řazení technikou rozděl a panuj

- Vyvážené dělení vstupní posloupnosti na přibližně stejně velké části:
 - Důraz kladen na analýzu vstupu – snažíme se rozdělít vstupní posloupnost na dvě přibližně stejně velké části, kde první obsahuje data, která jsou všechna menší než data druhé skupiny – řazení dělením (**QUICK-SORT**).
 - Důraz kladen na syntézu výstupu – vstupní posloupnost rozdělíme bez rozmýšlení na dvě přibližně stejně velké části. Při syntéze slučujeme seřazené posloupnosti do výsledku – řazení slučováním (**MERGE-SORT**).
- Nevyvážené dělení vstupní posloupnosti na část obsahující jeden prvek a zbytek:
 - Důraz kladen na analýzu vstupu – snažíme se ve vstupní posloupnosti najít minimální prvek, který vložíme do čela výstupní posloupnosti – řazení přímým výběrem (**SELECTION-SORT**).
 - Důraz kladen na syntézu výstupu – ze vstupní posloupnosti oddělíme bez rozmýšlení první prvek, který při syntéze zařadíme do výsledku na správné místo – řazení zatřídováním (**INSERTION-SORT**).

Př.: Řazení slučováním (MERGE-SORT)

Rekurzivní řešení - potřebujeme seřadit obsah pole A mezi indexy p a r

1. Najdeme přibližný střed q a rozdělíme A na dvě části
2. Rekurzivně seřadíme obě části
3. Seřazené části sloučíme

MERGE-SORT(A, p, r)

1. **if** $p < r$ **then** {
2. $q = \lfloor (p + r)/2 \rfloor$;
3. MERGE-SORT(A, p, q);
4. MERGE-SORT($A, q + 1, r$);
5. MERGE(A, p, q, r)
6. }

Časová složitost $T(n)$:

Pro $n=1$ – konstantní

Pro $n>1$ – 2-krát seřadit poloviny + sloučení

$$T(n) = \begin{cases} \Theta(1) \\ 2T(n/2) + \Theta(n) \end{cases}$$

Tj. $T(n) = \Theta(n \ln n)$ (odvození je mimo PRG2)

Typy rekurze

- Lineární rekurze: V těle algoritmu je pouze jedno rekurzivní volání anebo jsou dvě, ale vyskytují se v disjunktních větvích podmíněných příkazů a nikdy se neprovedou současně.
- Př.: Faktoriál

$$\text{FAC}(n) = 1 \quad \text{if } n = 1,$$

$$\text{FAC}(n) = n * \text{FAC}(n - 1) \quad \text{if } n > 1.$$

- Koncová rekurze (tail recursion): rekurzivní volání je posledním příkazem algoritmu, po kterém se už neprovádějí žádné další "odložené" operace.
- Př.: Největší společný dělitel (Euclidův algoritmus): Nechť $n \geq m \geq 0$.

$$\text{GCD}(n, m) = n \quad \text{if } m = 0,$$

$$\text{GCD}(n, m) = \text{GCD}(m, \text{REMAINDER}(n, m)) \quad \text{if } m > 0.$$

Typy rekurze (pokr.)

- Kaskádní rekurze: V těle algoritmu jsou vedle sebe aspoň dvě rekurzivní volání.
Viz MERGESORT, QUICKSORT.

- Příklad: Fibonacciho čísla

$$F(n) = 1 \quad \text{if } n = 1, 2$$

$$F(n) = F(n - 1) + F(n - 2) \quad \text{if } n > 2.$$

- Vnořená rekurze: Rekurzivní funkce, jejíž argumenty jsou specifikovány rekurzivně.

- Příklad: Ackermannova funkce

$$A(m, n) = n + 1 \quad \text{if } m = 0,$$

$$A(m, n) = A(m - 1, 1) \quad \text{if } m > 0 \text{ and } n = 0,$$

$$A(m, n) = A(m - 1, A(m, n - 1)) \quad \text{if } m > 0 \text{ and } n > 0.$$

- Neuvěřitelně rychle rostoucí funkce, viz

<http://mathworld.wolfram.com/AckermannFunction.html>.

Rekurze vs. iterace

- Rekurzivní programování má základní oporu v teoretické informatice, kde bylo dokázáno, že:
 1. Každou funkci, která může být implementovaná na vonNeumannovském počítači, lze vyjádřit rekurzivně bez použití iterace.
 2. Každá rekurzivní funkce se dá vyjádřit iterativně s použitím zásobníku (implicitní zásobník se stane viditelný uživateli).
- Koncovou rekurzi lze vždy nahradit iterací bez nutnosti zásobníku. Ta bývá zpravidla efektivnější.
- V rekurzivní proceduře se při rekurzivním volání téhož podprogramu (procedury nebo funkce) musí na systémový zásobník uložit hodnoty parametrů a lokálních proměnných volajícího podprogramu.
- Při návratu po ukončení tohoto vnořeného volání se tyto hodnoty obnoví.
- Výška zásobníku se rovná hloubce stromu rekurzivních volání procedury.
- Rekurzivní výpočet má tedy skrytou paměťovou náročnost úměrnou hloubce tohoto stromu.

Př.: Binární hledání (BINARY-SEARCH)

Zadání:

- Procedura BINARY-SEARCH prochází seřazené pole A v zadaném rozsahu $A[low..high]$ a hledá zadanou hodnotu v . Vrací index nalezené hodnoty, nebo NIL jako příznak, že hodnota v v daném úseku nebyla nalezena.
- Označení jako binární hledání je založeno na myšlence, že obsah pole A lze rozdělit na dvě části a podle hodnoty v místě dělení hledat dále jen v jedné z těchto částí (také se mluví o hledání binárním půlením).
- Zkusíme navrhnout rekurzivní i iterativní verzi tohoto algoritmu jako odpovídající procedury RECURSIVE-BINARY-SEARCH a ITERATIVE-BINARY-SEARCH s parametry BINARY-SEARCH($A, v, low, high$), kde A je prohledávané pole, v je hledaná hodnota a $low..high$ jsou indexy vymezující prohledávaný úsek.
- Naivní řešení hrubou silou je postupné procházení pole A v rozsahu $low..high$ a porovnávání $A[i]$ s hodnotou v . Složitost této verze bude zřejmě $\Theta(n)$, neboť v nejhorším případě musím projít úsek celý a porovnat všechny prvky. Zkusme najít řešení efektivnější.

Naivní verze SEARCH

```
int NaiveSearch(int A[], int v, int low, int high) {  
    int mid = low;  
    while (mid <= high) {  
        if (v == A[mid]) return mid;  
        else mid++;  
    };  
    return -1;  
};
```

- Procedura vždy skončí (mid dosáhne high).
- Složitost je: $T(n) = \Theta(n)$.



Rekurzivní verze BINARY-SEARCH

```
int RecursiveBinarySearch(int A[], int v, int low, int high) {  
    if (low > high) { return 0; }  
    int mid = (int)ceil((high-low)/2);  
    if (v == A[mid]) return mid;  
    else if (v > A[mid])  
        return RecursiveBinarySearch(A,v,mid+1,high);  
    else  
        return RecursiveBinarySearch(A,v,low,mid);  
};
```

- Procedura vždy skončí (rekurze pracuje vždy s menším a menším rozsahem).
- Složitost je dána: $T(n) = T(n/2) + \Theta(1) = \Theta(\lg n)$.

Iterativní verze BINARY-SEARCH

```
int IterativeBinarySearch(int A[], int v, int low, int high) {  
    int mid;  
    while (low <= high) {  
        mid = (int)ceil((high-low)/2);  
        if (v == A[mid]) return mid;  
        else if (v > A[mid]) low = mid+1;  
        else high = mid-1;  
    };  
    return -1;  
};
```

- Procedura vždy skončí (indexy v iteraci se přibližují).
- Složitost je dána: $T(n) = T(n/2) + \Theta(1) = \Theta(\lg n)$.

Kdy brutální síla vítězí?

Existují případy, kdy brutální síla vítězí. Jinými slovy naivní algoritmus může být někdy úspěšným řešením. Jako příklad uvažme hru „[Master Mind](#)“. Brutální síla je zde efektivní, neboť řešení, které probíhá všechny možnosti a vyškrtává neplatné kombinace vede k cíli přímočaře a pravděpodobně nejrychleji. Sofistikované algoritmy pro procházení stavového prostoru možností tak úspěšné nebývají. Je to ale tím, že rozsah stavového prostoru je poměrně malý – variace K prvků třídy $N = N^K$. Pro $N=4$ a $K=6$ tedy 1296 možných variací. Maximální počet pokusů je ≤ 5 .



[Donald Knuth](#) (1977) navrhl algoritmus:

1. Vytvoř množinu S zbývajících možností (na začátku 1296). Prvý odhad bude aabb
2. Vyhoď z S všechny možnosti, které neodpovídají ohodnocení.
3. Pro všechny možné odhady (nemusí být v S) spočti, kolik budou eliminovat prvků z S – to je jeho skóre a vyber vždy odhad s nejmenším skóre.
4. Jako další odhad použij ten, který má největší skóre (minimax).
5. Pokud jsi neuhodl, jdi na krok 2.

The End